

Reviewer search cheatsheet (1/5)

Tick = examined. Use the full checklist for detailed checks, applicability and source pointers.

☐ T00 Establish the submission and evidence scope

Where: Cover; index; amendments • **Search:** module, revision, attachment

Why: Prevents reviewing the wrong snapshot.

Details to check: Device/functions; pathway; governing revisions; accessible attachments; prior evidence; open commitments.

☐ T01 Check the documentation/classification rationale

Where: Level evaluation; risk file; architecture • **Search:** Basic, Enhanced, safety class

Why: Determines which evidence route applies.

Details to check: Level and safety class separately; harm scenarios; item boundaries; external controls; rationale.

☐ T02 Find what the software does and where it operates

Where: Description; SRS; IFU; interfaces • **Search:** input, output, workflow, platform

Why: Defines relevant risks and test conditions.

Details to check: Users/population; functions; input/output units and limits; platforms; external and other-function interactions.

☐ T03 Match proposed, tested and labeled versions

Where: Manifest; test headers; IFU • **Search:** build, baseline, version

Why: Results must relate to the candidate release.

Details to check: Version/build; dependencies/settings; user-visible identity; differences; supporting bridge or retests.

☐ T04 Follow hazards through verified risk controls

Where: Risk file; trace matrix; tests • **Search:** harm, control, residual

Why: Finds unsupported risk-reduction arguments.

Details to check: Scenario/harm; criteria; control; implementation AND effectiveness evidence; new risks; individual/overall residual risk.

☐ T05 Check requirements are specific and verifiable

Where: SRS; trace matrix; test records • **Search:** shall, threshold, verifies

Why: Tests need defined targets and coverage.

Details to check: Current IDs; behavior/conditions; measurable limits; safety/security links; design links; executed verification; changes.

☐ T06 Trace system boundaries and interfaces

Where: Architecture; deployment/interface diagrams • **Search:** module, interface, segregation

Why: Exposes hidden dependencies and connections.

Details to check: Responsibilities; data direction; external services; shared resources; enforced segregation; release consistency.

Read original footnotes and exceptions; compare the same version and conditions.

Reviewer search cheatsheet (2/5)

Tick = examined. Use the full checklist for detailed checks, applicability and source pointers.

☐ T07 Check the applicable design evidence

Where: SDS; design reviews; trace matrix · **Search:** design, algorithm, implements

Why: Connects requirements to implementation choices.

Details to check: Logic/interfaces; exceptional states; requirement links; review findings; revisions. Detailed SDS is not a routine Basic-level request.

☐ T08 Check development and change-control arrangements

Where: Process description; plans; conformity claims · **Search:** lifecycle, change control, maintenance

Why: Supports controlled, reproducible software evolution.

Details to check: Applicable evidence route; baselines; tools/coding; traceability; regression; problem resolution; legacy gaps when relevant.

☐ T09 Check what was tested and actually observed

Where: Protocols; results; logs; regression reports · **Search:** criterion, actual result, failed, retest

Why: A Pass label alone may conceal missing observations.

Details to check: Build/conditions; expected versus actual; units; coverage/counts; failures; fixes/retests; regression rationale; release limitations.

☐ T10 Check the bridge from earlier to current software

Where: Version history; impact analysis; reruns · **Search:** change, last tested, equivalent

Why: Earlier evidence may not cover changed behavior.

Details to check: Version sequence; changed items; risk/performance effects; reuse rationale; repeated tests; study/authorized versions.

☐ T11 Check open anomalies and their dispositions

Where: Anomaly list; failures; release review · **Search:** unresolved, workaround, deferred

Why: Open defects can qualify release conclusions.

Details to check: Snapshot/counts; IDs and triggers; affected versions; safety/security impact; rationale; workaround; user communication.

☐ T12 Check externally developed components

Where: OTS/SOUP review; SBOM; supplier records · **Search:** component version, known anomaly, support

Why: Supplier evidence may not cover device-specific use.

Details to check: Exact identity; role/limits; environment; anomaly review; device tests; permitted configurations; support/replacement.

Read original footnotes and exceptions; compare the same version and conditions.

Reviewer search cheatsheet (3/5)

Tick = examined. Use the full checklist for detailed checks, applicability and source pointers.

□ T13 Check the security-risk process and conclusion

Where: Security plan/report; safety-risk links · **Search:** scope, criteria, residual risk

Why: An overall statement can hide excluded systems.

Details to check: Covered assets/configuration; method; criteria; roles/suppliers; safety interface; open findings; conclusion boundaries.

□ T14 Follow modeled attack paths and assumptions

Where: Threat model; data flows; control/test links · **Search:** threat, entry point, trusted network

Why: Reveals unmodeled paths and unsupported assumptions.

Details to check: System/method; trust boundaries; attacker prerequisites; lifecycle states; exclusions; validated/transferred assumptions; mitigations/tests.

□ T15 Check device-specific security-risk decisions

Where: Risk worksheets; controls; retests · **Search:** CVSS, rubric, mitigation, transfer

Why: Scores alone do not demonstrate risk reduction.

Details to check: Affected version; scoring basis; pre/post assumptions; control effectiveness; residual/new risks; external reliance.

□ T16 Check SBOM identity and coverage

Where: SBOM; manifests; architecture · **Search:** dependency, supplier, version, excluded

Why: Vulnerability review depends on an accurate component scope.

Details to check: Product/build; format/date; component identifiers; direct/transitive dependencies; system exclusions; support and vulnerability links.

□ T17 Check vulnerabilities and component support

Where: Scan/CVE mapping; risk review; lifecycle notices · **Search:** KEV, query date, not exploitable

Why: Empty scans or low scores can conceal exposure.

Details to check: Search scope/date; installed-version match; exploitability basis; device impact; mitigation/retest; fielded versions; support gaps.

□ T18 Check security consequences of open anomalies

Where: Anomaly security review; vulnerability register · **Search:** weakness, security impact, CWE

Why: Functional acceptance does not settle exploitability.

Details to check: Same list snapshot/IDs; attacker-triggered behavior; vulnerability rationale; linked control; verification; residual consequences.

Read original footnotes and exceptions; compare the same version and conditions.

Reviewer search cheatsheet (4/5)

Tick = examined. Use the full checklist for detailed checks, applicability and source pointers.

☐ T19 Follow security links through executed results

Where: Trace exports; architecture; test runs · **Search:** threat ID, control ID, run ID

Why: A link to a planned test does not show execution.

Details to check: Compatible IDs/versions; threat-to-control chain; component links; run outcomes; reverse coverage; dangling targets.

☐ T20 Check metric definitions and actual populations

Where: Metrics; patch register; deployment records · **Search:** denominator, identified, deployed

Why: Patch release and field installation measure different things.

Details to check: Period/version scope; numerator/denominator; timing events; units; zero versus unavailable; comparison with commitments.

☐ T21 Check mechanisms behind security-control labels

Where: Security design; specifications; control tests · **Search:** authentication, key, log, recovery

Why: Control names do not show protection or effectiveness.

Details to check: Identity/roles; crypto/key lifecycle; integrity/confidentiality; logs; safe failure; backup; signed/interrupted updates; verification.

☐ T22 Trace security architecture end to end

Where: Global/multi-patient/update/use-case views · **Search:** endpoint, port, trust boundary

Why: Protection may stop at an intermediary or different state.

Details to check: Ownership; shared services; update path; operating states; protocols/direction; defaults; authentication/crypto; control/test links.

☐ T23 Check security tests and remaining findings

Where: Original test/penetration reports; retests · **Search:** exclusion, fuzz, independent, accepted

Why: Methods and boundaries determine what results demonstrate.

Details to check: Build/tools/dates; threat/requirement scope; tester expertise/independence; actual findings; fixes/retests; accepted or deferred residuals.

☐ T24 Find actionable security instructions

Where: IFU; administrator guide; support notices · **Search:** ports, incident, SBOM, sanitize

Why: User-dependent controls need usable instructions.

Details to check: Deployment controls; version/SBOM access; logs/response; backup/recovery; support dates; risky changes; transferred risks; retirement.

Read original footnotes and exceptions; compare the same version and conditions.

Reviewer search cheatsheet (5/5)

Tick = examined. Use the full checklist for detailed checks, applicability and source pointers.

☐ T25 Check postmarket security commitments

Where: Management/PSIRT plan; disclosure/update policy • **Search:** monitoring, patch, fielded, end of service

Why: Protection depends on sustained detection and response.

Details to check: Owners; sources/frequency; disclosure intake; routine/exceptional triggers; timelines; deployment; support transition; completed evidence.

☐ T26 Check cyber-device applicability facts

Where: Applicability statement; connectivity description • **Search:** 524B, internet, software

Why: The decision needs factual and legal support.

Details to check: Software characteristic; direct/indirect connectivity capability; vulnerable technology; rationale; relevant monitoring/update/SBOM evidence.

☐ T27 Check use-related risks and user evidence

Where: Use specification; task analysis; HF report • **Search:** critical task, close call, assistance

Why: Task success may conceal unsafe or assisted use.

Details to check: Applicable category; users/UI/training; critical tasks; study conditions; errors/difficulties; design changes; residual-risk rationale.

☐ T28 Check the exact standards claim

Where: Declaration; recognition entry; original reports • **Search:** edition, clause, deviation, scope

Why: A standard title does not establish conformity.

Details to check: Product/edition/options; dated recognition; exclusions; evidence route; completed results; tested-product bridge; lab scope where relied on.

☐ T29 Match labeling to tested behavior and risks

Where: IFU; version screen; risk/validation records • **Search:** version, limitation, warning

Why: Instructions and claims must support the relevant release.

Details to check: Intended use/limits; software version; permitted components; anomaly/residual-risk disclosures; effectiveness of user controls; network conditions.

☐ T30 Check the model, data and validation chain

Where: Model/data records; results; labeling; PCCP • **Search:** partition, subgroup, uncertainty, modification

Why: Performance is specific to an artifact and population.

Details to check: Model/pipeline; data provenance; test independence; reference labels; targets/results; subgroup limits; controls; PCCP status/scope if used. AI lifecycle draft: background only.

Read original footnotes and exceptions; compare the same version and conditions.